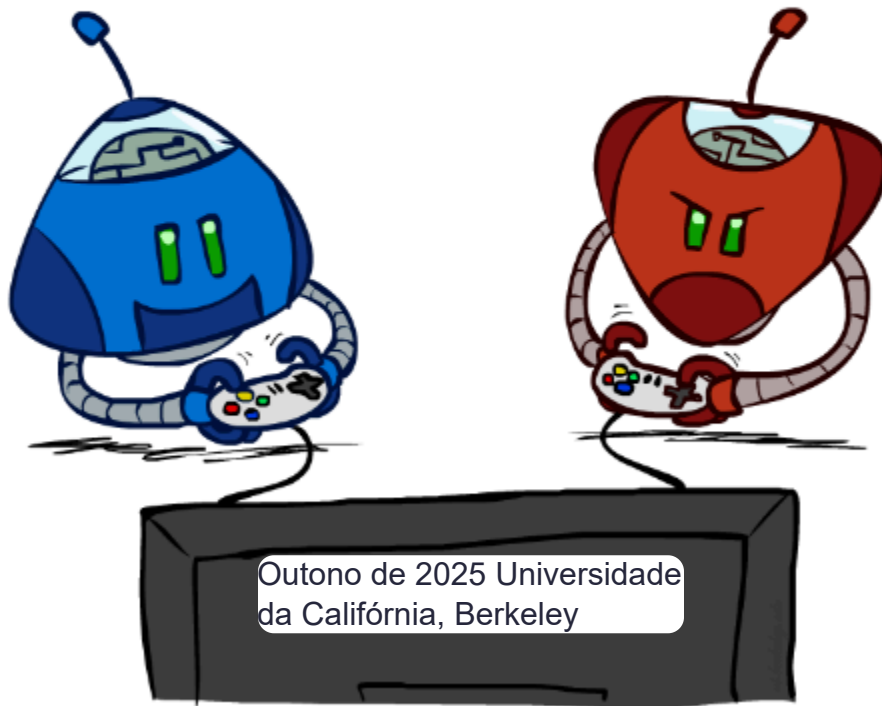


CS 188: Inteligência Artificial

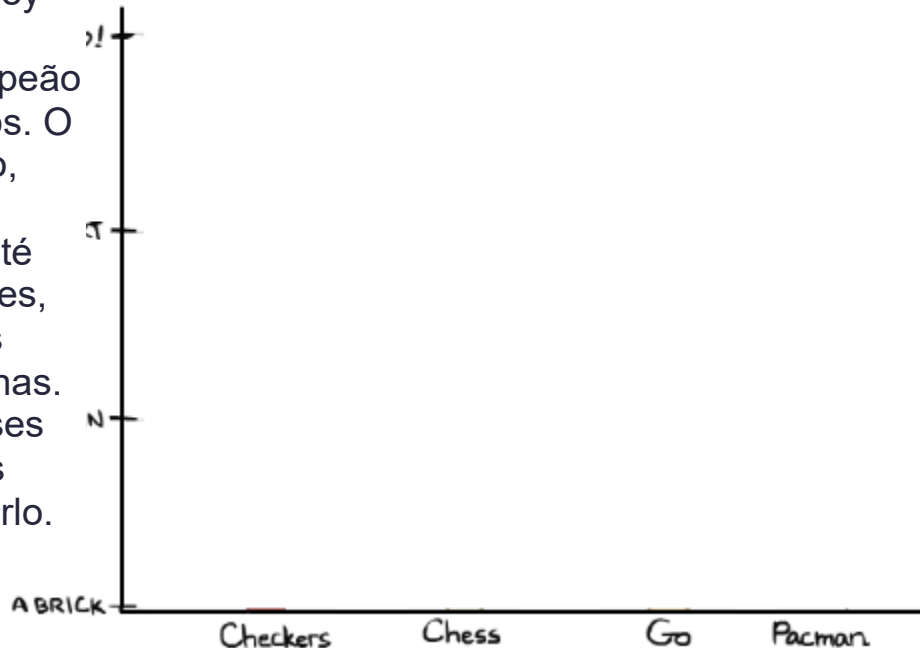
Busca Adversarial



Outono de 2025 Universidade
da Califórnia, Berkeley

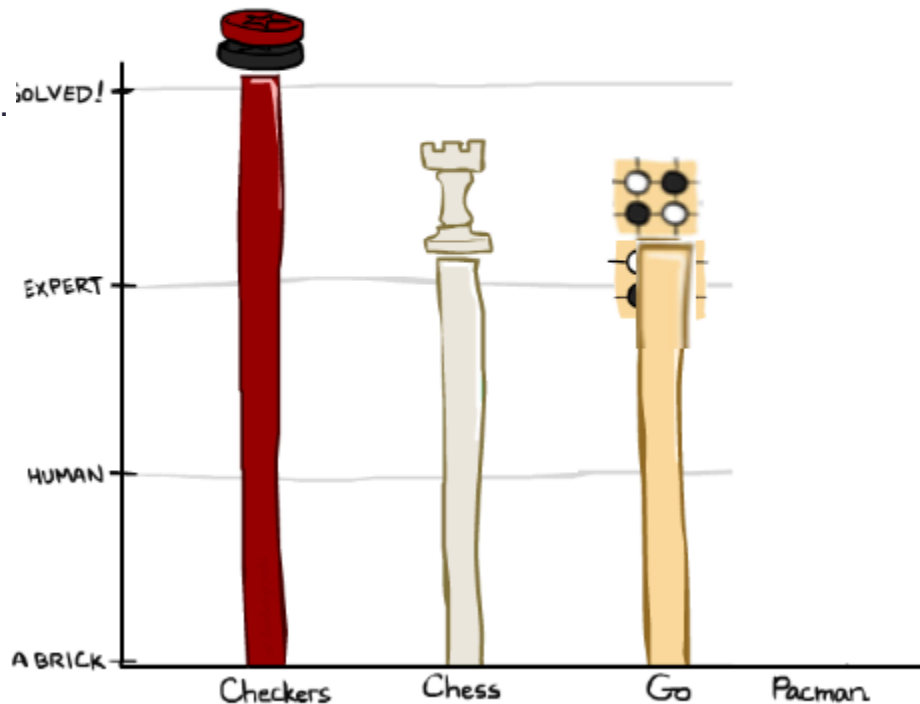
Jogos de ponta

§ Damas: 1950: Primeiro jogador de computador. 1994: Primeiro campeão computadorizado: Chinook encerrou o reinado de 40 anos da campeã humana Marion Tinsley usando um final completo de 8 peças. 2007: Damas resolvidas! § Xadrez: 1997: Deep Blue derrota o campeão humano Gary Kasparov em uma partida de seis jogos. O Deep Blue examinou posições de 200M por segundo, utilizou avaliação muito sofisticada e métodos não divulgados para estender algumas linhas de busca até 40 camadas. Os programas atuais são ainda melhores, embora menos históricos. § Go: Campeões humanos agora estão começando a ser desafiados por máquinas. Em jogo, $b > 300$! Os programas clássicos usam bases de conhecimento de padrões, mas avanços recentes usam métodos de expansão (aleatória) do Monte Carlo.

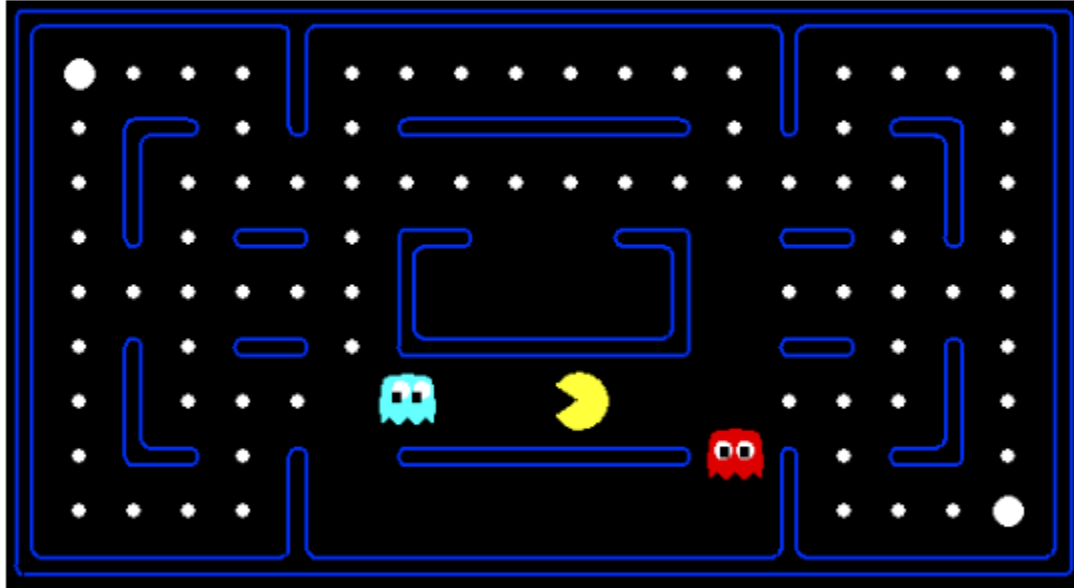


Jogos de ponta

§ Damas: 1950: Primeiro jogador de computador.
1994: Primeiro campeão computadorizado: Chinook encerrou o reinado de 40 anos da campeã humana Marion Tinsley usando um final completo de 8 peças.
2007: Damas resolvidas! § Xadrez: 1997: Deep Blue derrota o campeão humano Gary Kasparov em uma partida de seis jogos. O Deep Blue examinou posições de 200M por segundo, utilizou avaliação muito sofisticada e métodos não divulgados para estender algumas linhas de busca até 40 camadas. Os programas atuais são ainda melhores, embora menos históricos. § Go: 2016: Alpha GO derrota campeão humano. Usa Busca em Árvore de Monte Carlo, função de avaliação aprendida. § Pacman



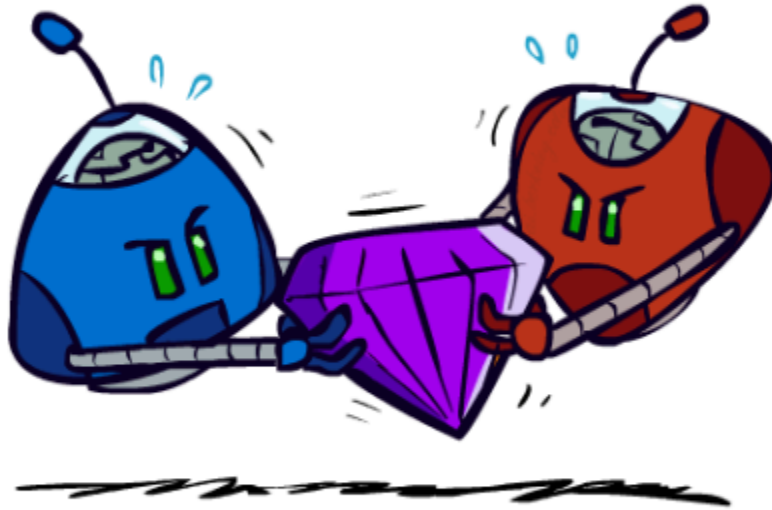
Comportamento a partir da computação



Vídeo da demo Mystery Pacman



Jogos Adversariais



Tipos de Jogos

§ Muitos tipos diferentes de jogos!

§ Machados:

§ Determinístico ou estocástico?
§ Um, dois ou mais jogadores?
§ Soma zero?
§ Informação perfeita (você consegue ver o estado)?



§ Quero algoritmos para calcular uma estratégia (política) que recomende um movimento de cada estado

Jogos Determinísticos

§ Muitas formalizações possíveis, uma é:

§ Estás: S (começa em s_0) § Jogadores:

$P = \{1 \dots N\}$ (geralmente se revezam) §

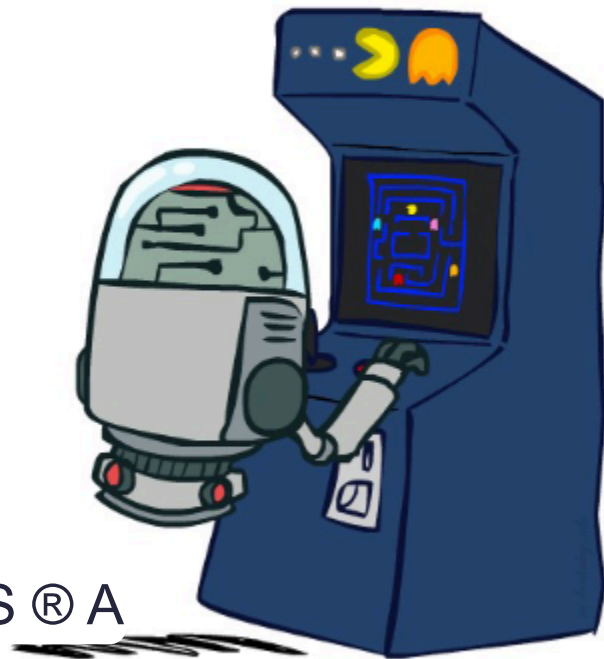
Ações: A (pode depender do jogador /

estado) § Função de transição: $S \times A \rightarrow$

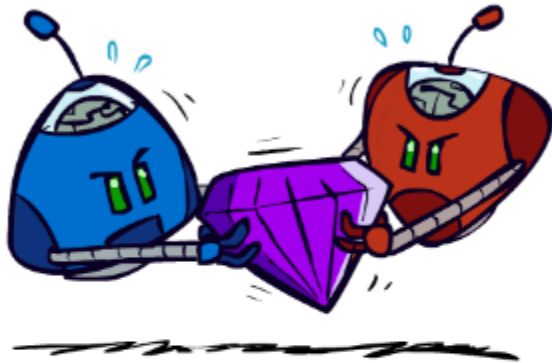
S § Teste Terminal: $S \rightarrow \{t, f\}$ § Utilidades

Terminais: $S \times P \rightarrow \mathbb{R}$

§ Solução para um jogador é uma política: $S \rightarrow A$



Jogos de soma zero



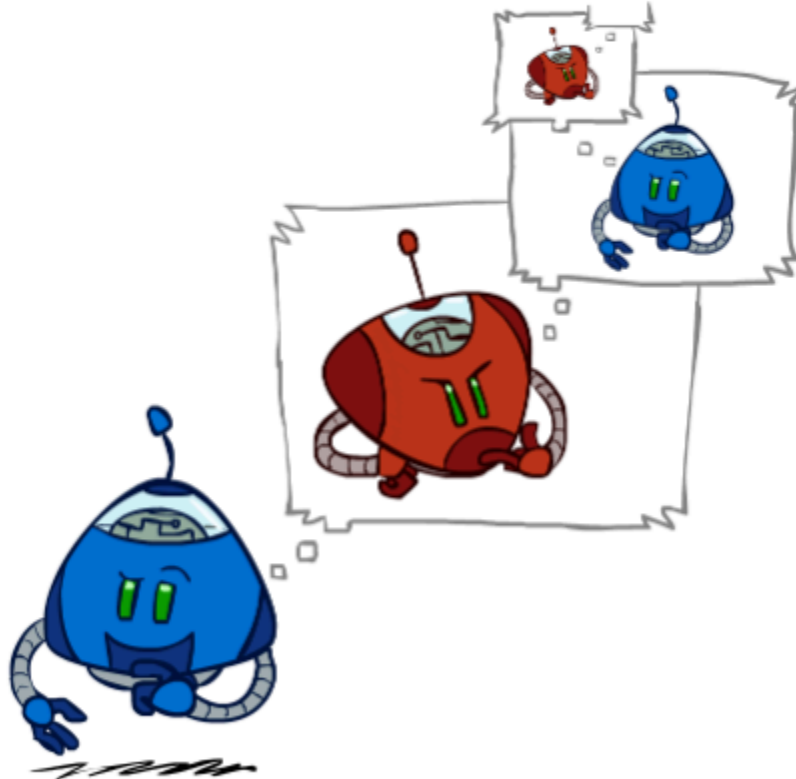
§ Jogos de soma zero

§ Agentes têm utilidades opostas (valores sobre resultados)
§ Vamos pensar em um único valor que um maximiza e o outro minimiza
§ Competição adversarial, pura

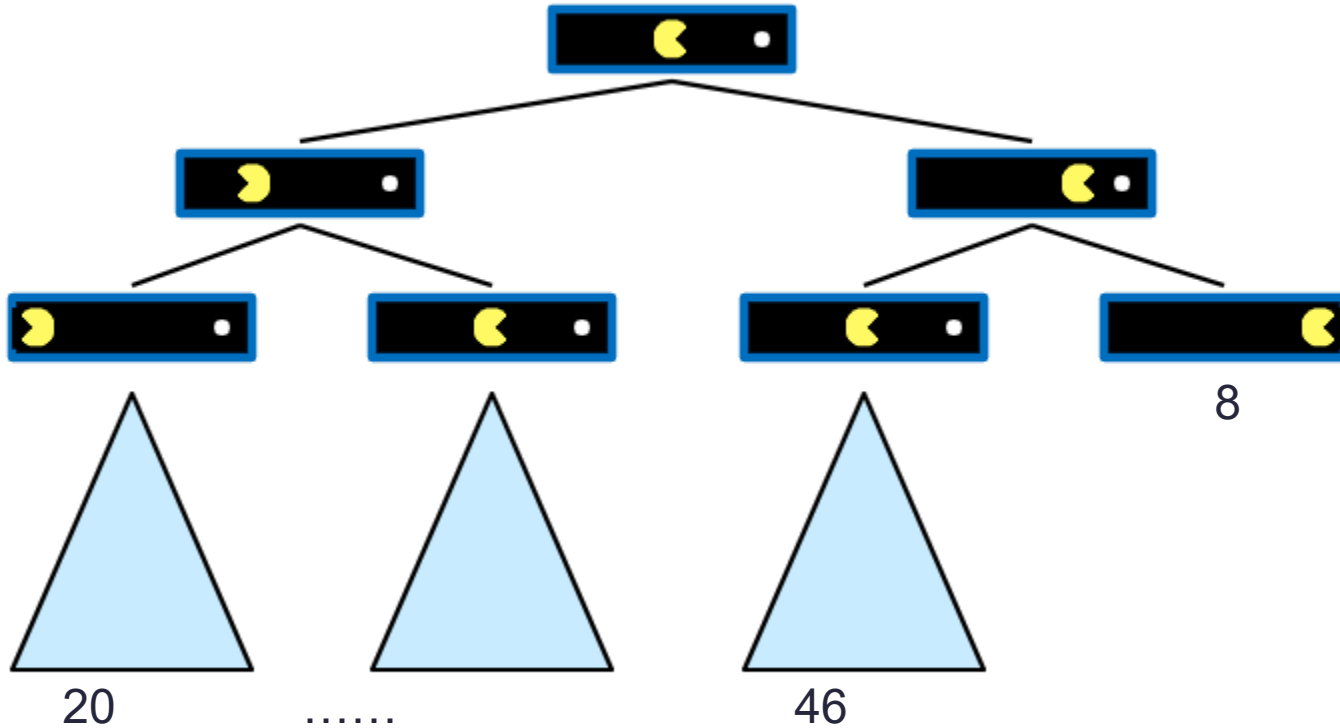
§ Jogos Gerais

§ Agentes têm utilidades independentes (valores sobre resultados)
§ Cooperação, indiferença, competição e mais são todas possíveis
§ Mais sobre jogos não de soma zero mais adiante

Busca Adversarial

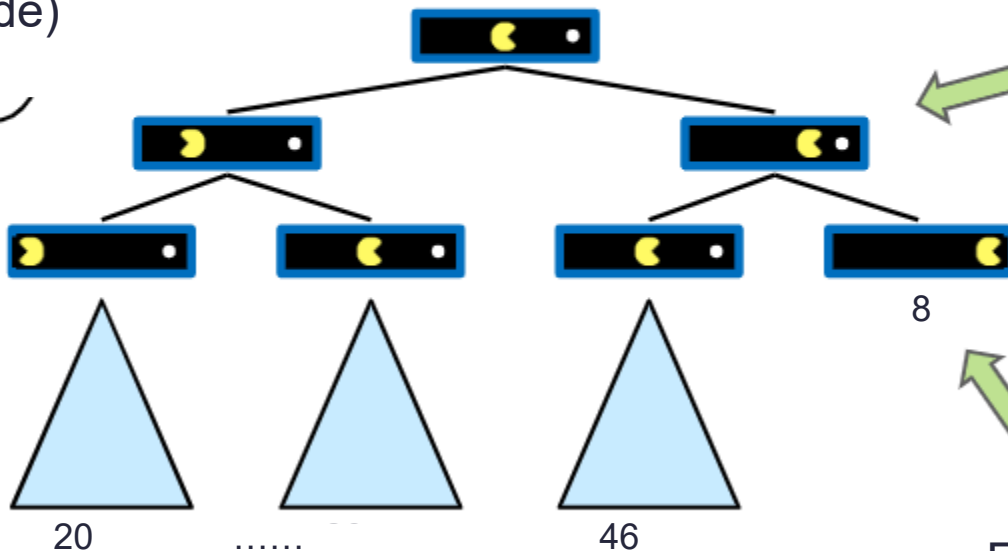


Árvores de Agente Único



Valor de um Estado

Valor de um estado:
O melhor resultado
alcançável (utilidade)
desse estado



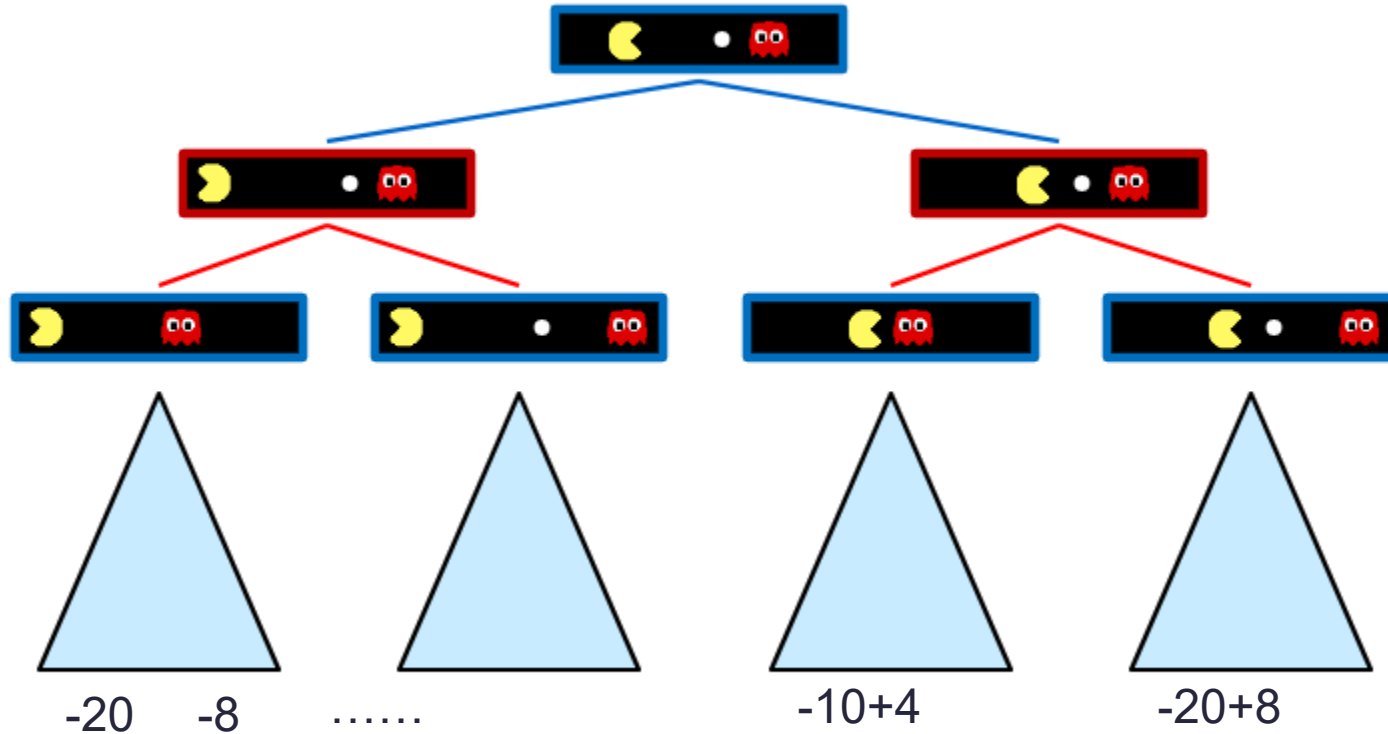
Estados Não Terminais:

$$V(s) = \max_{s' \in \text{children}(s)} V(s')$$

Estados Terminais:

$$V(s) = \text{known}$$

Árvores de Jogo Adversariais



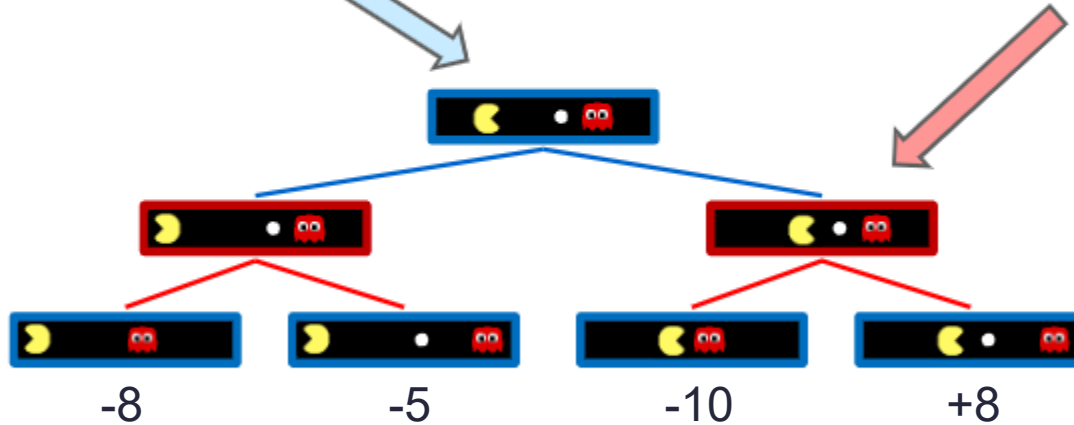
Valores Minimax

Estados sob Controle do Agente:

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

Estados sob controle do oponente:

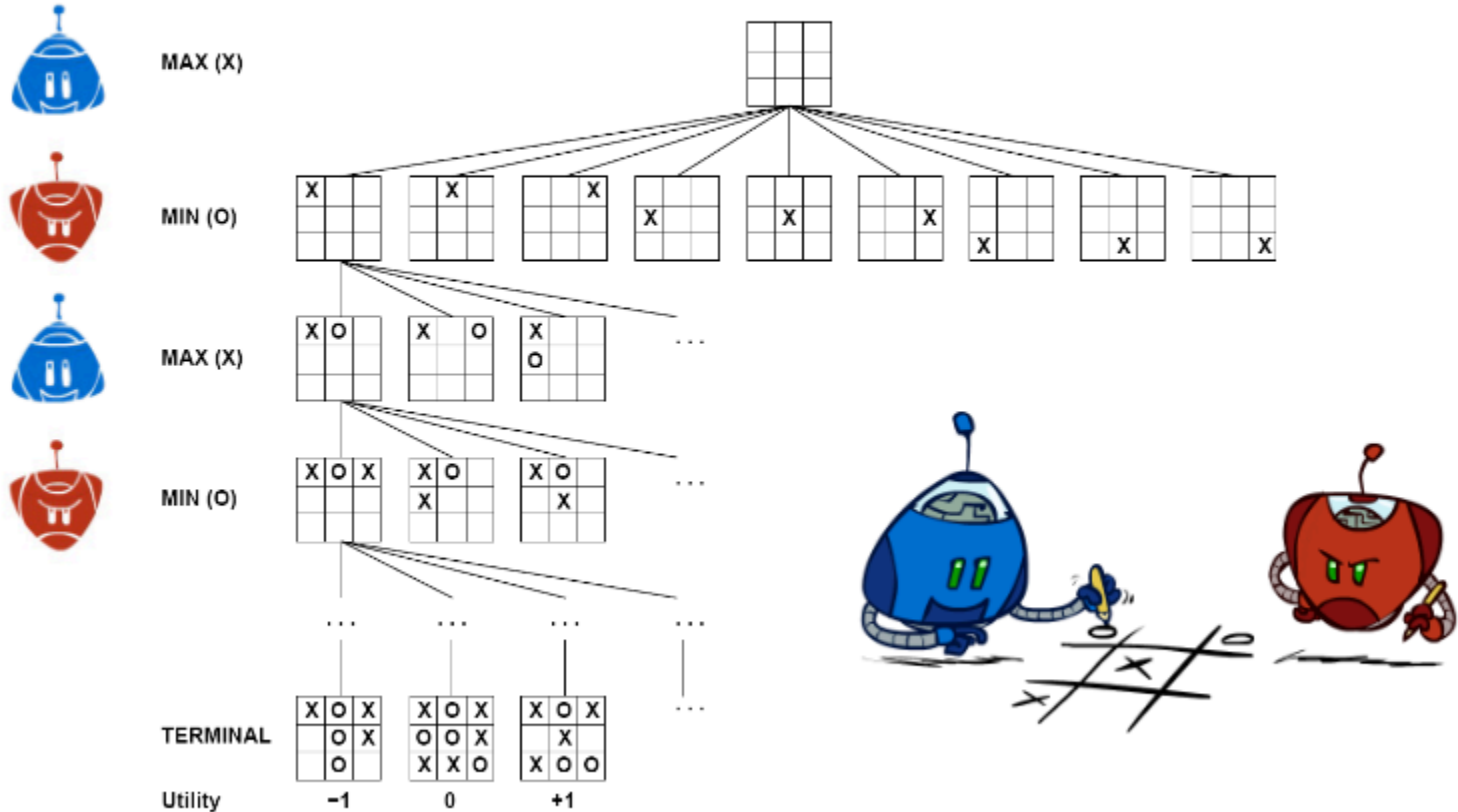
$$s \in \text{successors}(s')$$



Estados Terminais:

$$V(s) = \text{known}$$

Árvore de Jogo do Jogo da Velha



Busca Adversarial (Minimax)

§ Jogos determinísticos, soma zero:

§ Jogo da velha, xadrez, damas§

Um jogador maximiza o resultado§

O outro minimiza o resultado

§ Busca minimax:

§ Uma árvore de busca em espaço de

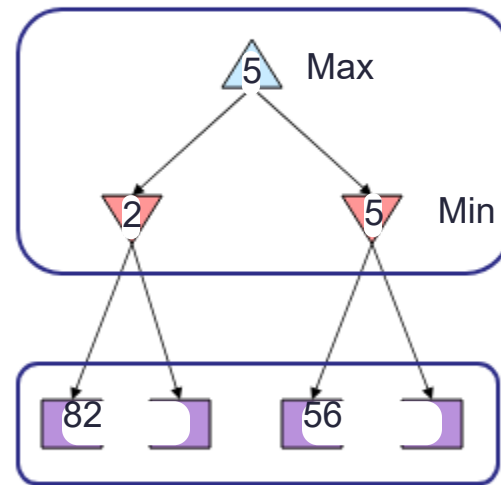
estados § Jogadores alternam turnos §

Calculam o valor minimax de cada nó:

a melhor utilidade alcançável contra um

Adversário racional (ótimo)

Valores minimax:
calculados recursivamente



Valores terminais:
parte do jogo

Implementação Minimax

```
def max-value(state):  
    initialize v =  
    -∞  
    para cada sucessor do state:  
        v =  
        max(v, min-value(sucessor))  
    retorne v
```



```
def min-value(state):  
    initialize v =  
    +∞  
    para cada sucessor do estado:  
        v =  
        min(v, max-  
        value(sucessor))  
    retorne v
```

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

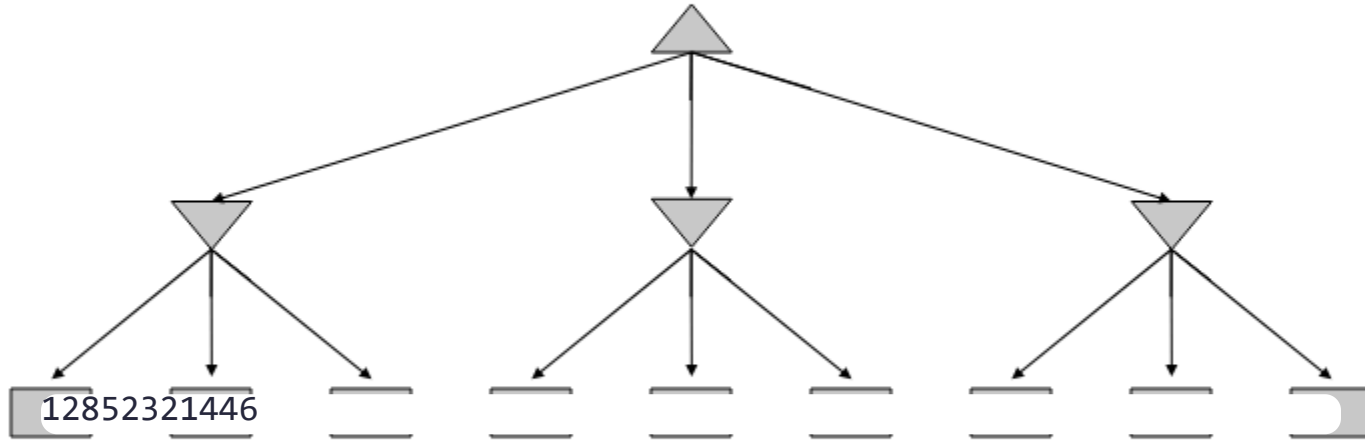
Implementação Minimax (Despacho)

```
def value(state): se o estado for um estado terminal:  
    devolver a utilidade do estado  
se o próximo agente for MAX: retorne max-value(estado)  
se o próximo agente for MIN: return min-value(state)
```

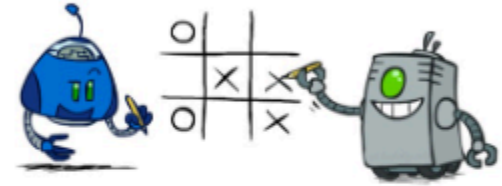
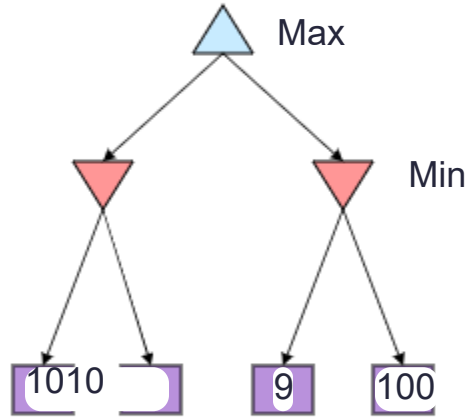
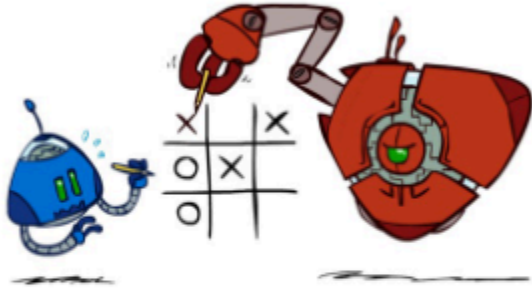
```
def max-value(state): initialize v =  $-\infty$   
para cada sucessor do state:  $v = \max(v, \text{value}(\text{sucessor}))$   
retorne v
```

```
def min-value(state): initialize v =  $+\infty$   
para cada sucessor do estado:  $v = \min(v, \text{valor}(\text{sucessor}))$   
retorne v
```

Exemplo Minimax



Propriedades Minimax



Ótimo contra um jogador perfeito. Caso contrário?

Vídeo de Demo Min vs. Exp (Min)



Vídeo de Demo Min vs. Exp (Exp)



Eficiência Minimax

§ Quão eficiente é o minimax?

§ Assim como

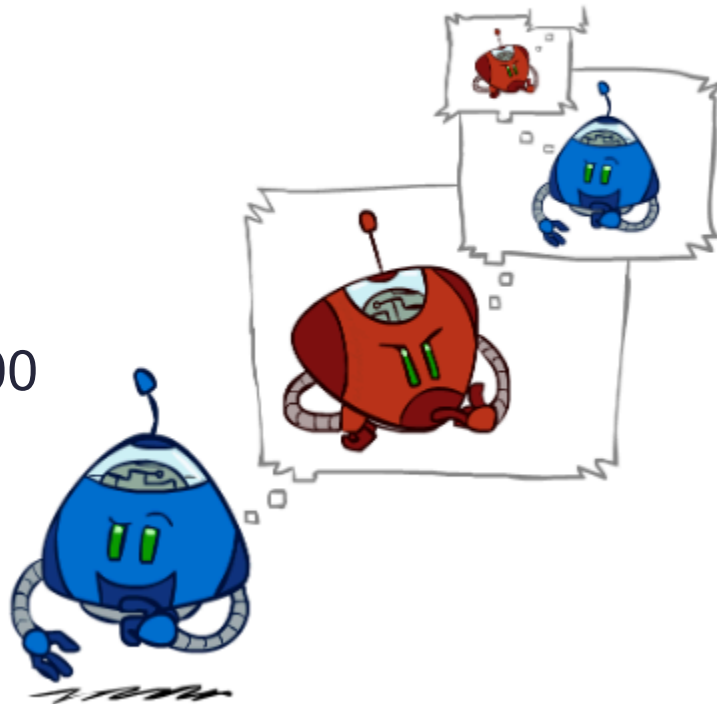
(exaustivo) DFS

§ Tempo: $O(bm)$

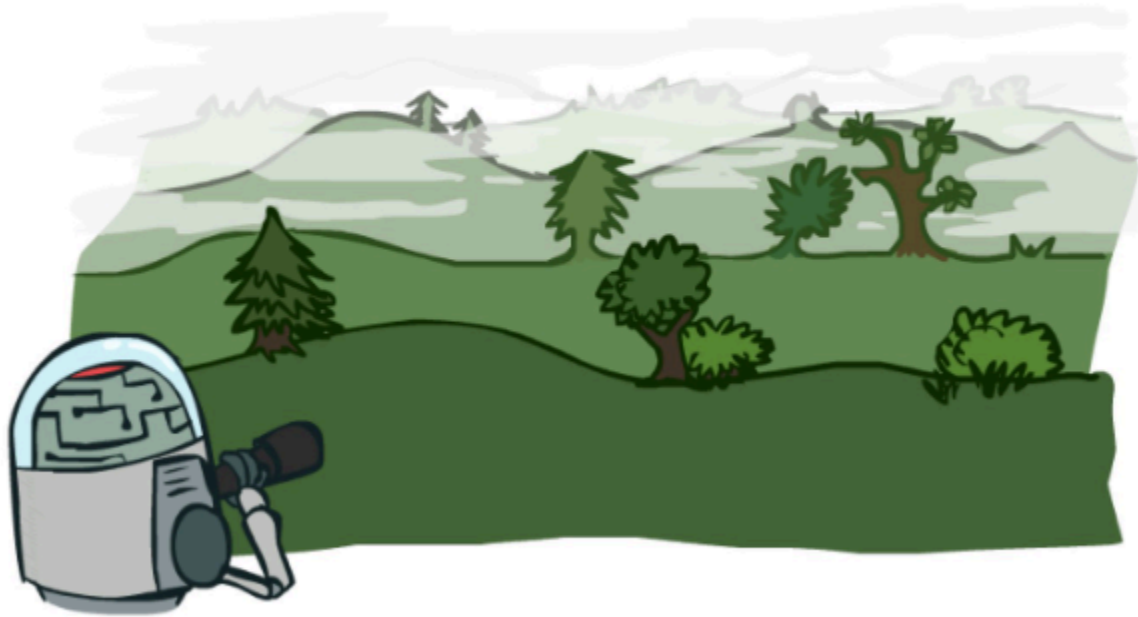
§ Espaço: $O(bm)$

§ Exemplo: Para xadrez, $b \gg 35$, $m \gg 100$

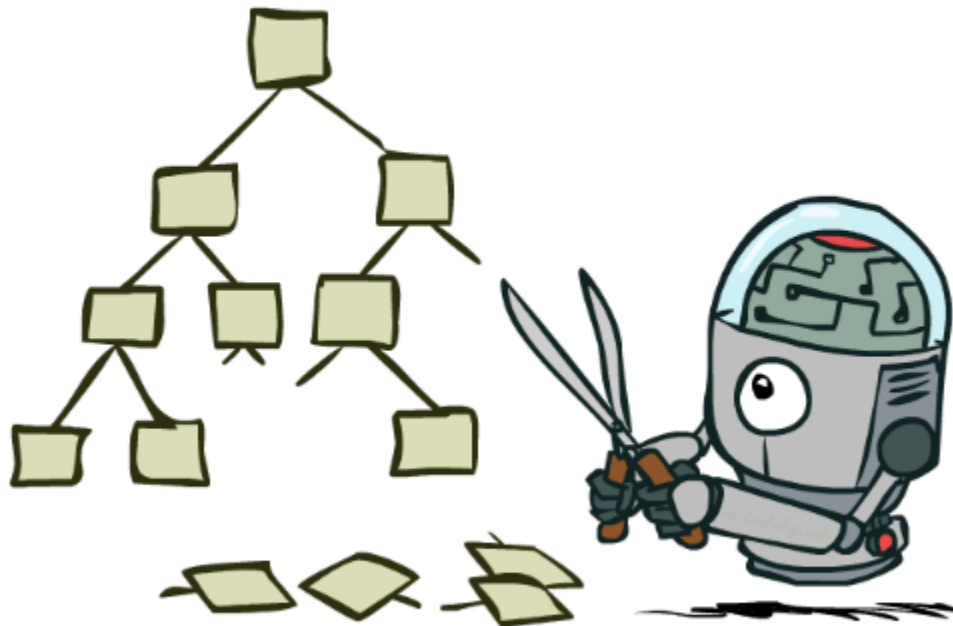
§ A solução exata é completamente inviável
§ Mas, precisamos explorar toda a árvore?



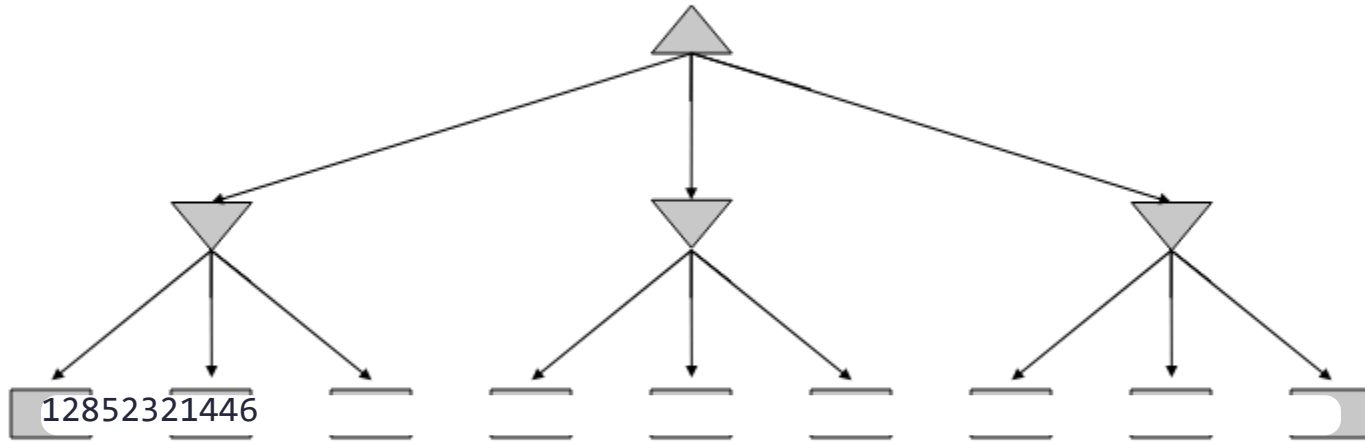
Limites de Recursos



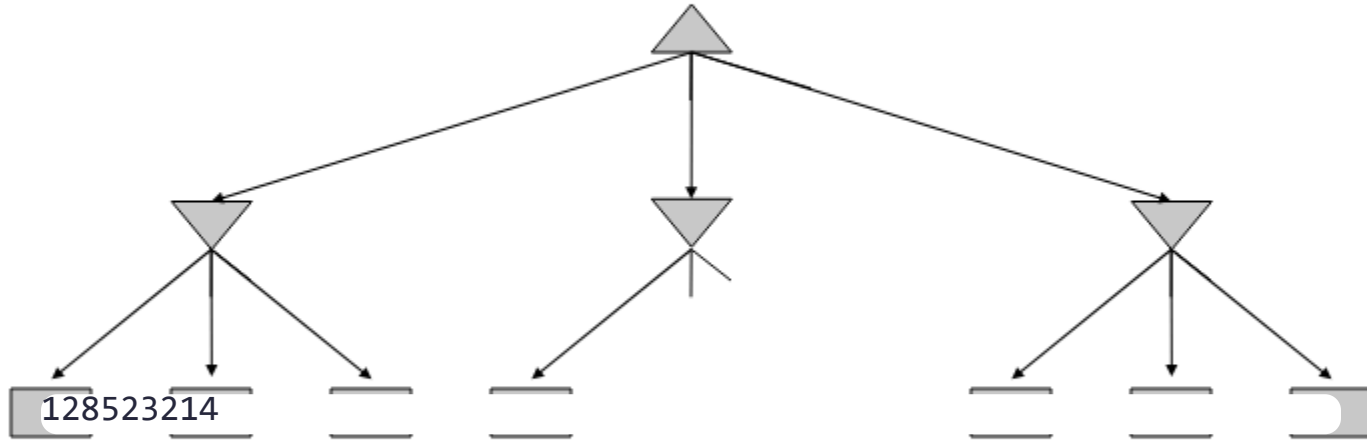
Poda de Árvores de Caça



Exemplo Minimax



Poda Minimax

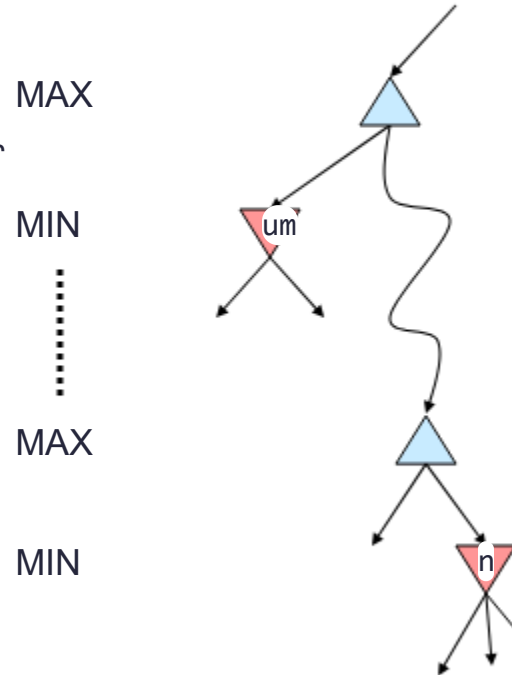


Poda Alfa-Beta

§ Configuração geral (versão MIN)

§ Estamos calculando o VALOR MÍNIMO em algum nó n § Estamos MAX fazendo looping sobre os filhos de n § a estimativa do valor mínimo das crianças por n está diminuindo § Quem se importa com o valor de n? MAX § Seja a melhor relação custo-benefício que o MAX pode obter em qualquer escolha ponto ao longo do caminho atual a partir da raiz § Se n ficar pior que a, MAX evitará isso, então podemos Pare de considerar os outros filhos do N (já é ruim o suficiente não ser jogado)

§ A versão MAX é simétrica



Implementação Alfa-Beta

α : Melhor opção do MAX para caminho para root
 β : Melhor opção do MIN para caminho para root

```
def max-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = -\infty$   
    para cada sucessor do estado:  
         $v = \max(v, \text{value}(\text{sucessor}, \alpha, \beta))$   
    se  $v \geq \beta$  devolver  $v$   
     $\alpha = \max(\alpha, v)$   
    retorne  $v$ 
```

```
def min-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = +\infty$   
    para cada sucessor do estado:  
         $v = \min(v, \text{value}(\text{sucessor}, \alpha, \beta))$   
    se  $v \leq \alpha$  return  $v$   
     $\beta = \min(\beta, v)$   
    return  $v$ 
```

Propriedades de poda alfa-beta

§ Essa poda não tem efeito sobre o valor minimax calculado para a raiz!

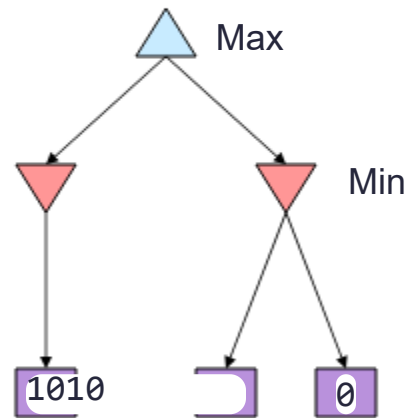
§ Os valores dos nós intermediários podem estar errados

§ Importante: filhos da raiz podem ter o valor errado § Então a versão mais ingênua não deixa você fazer seleção de ações

§ Uma boa organização infantil melhora a eficácia da poda

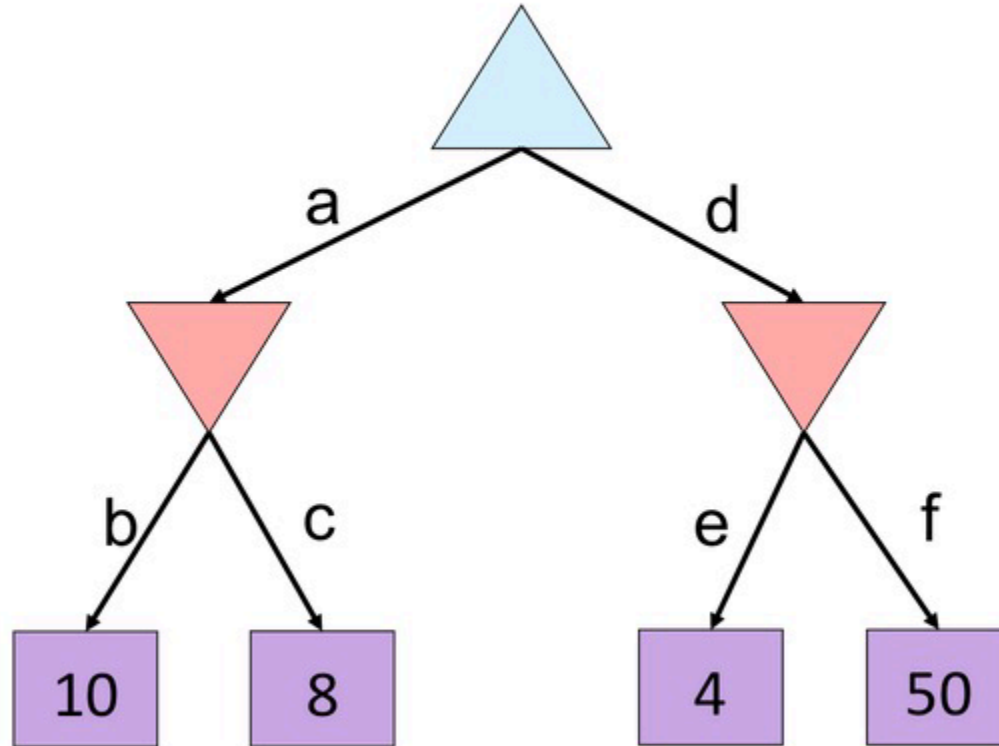
§ Com "ordem perfeita":

§ A complexidade temporal cai para $O(bm/2)$
Dobra a profundidade solúvel! § A busca completa, por exemplo, de xadrez, ainda é desesperada...

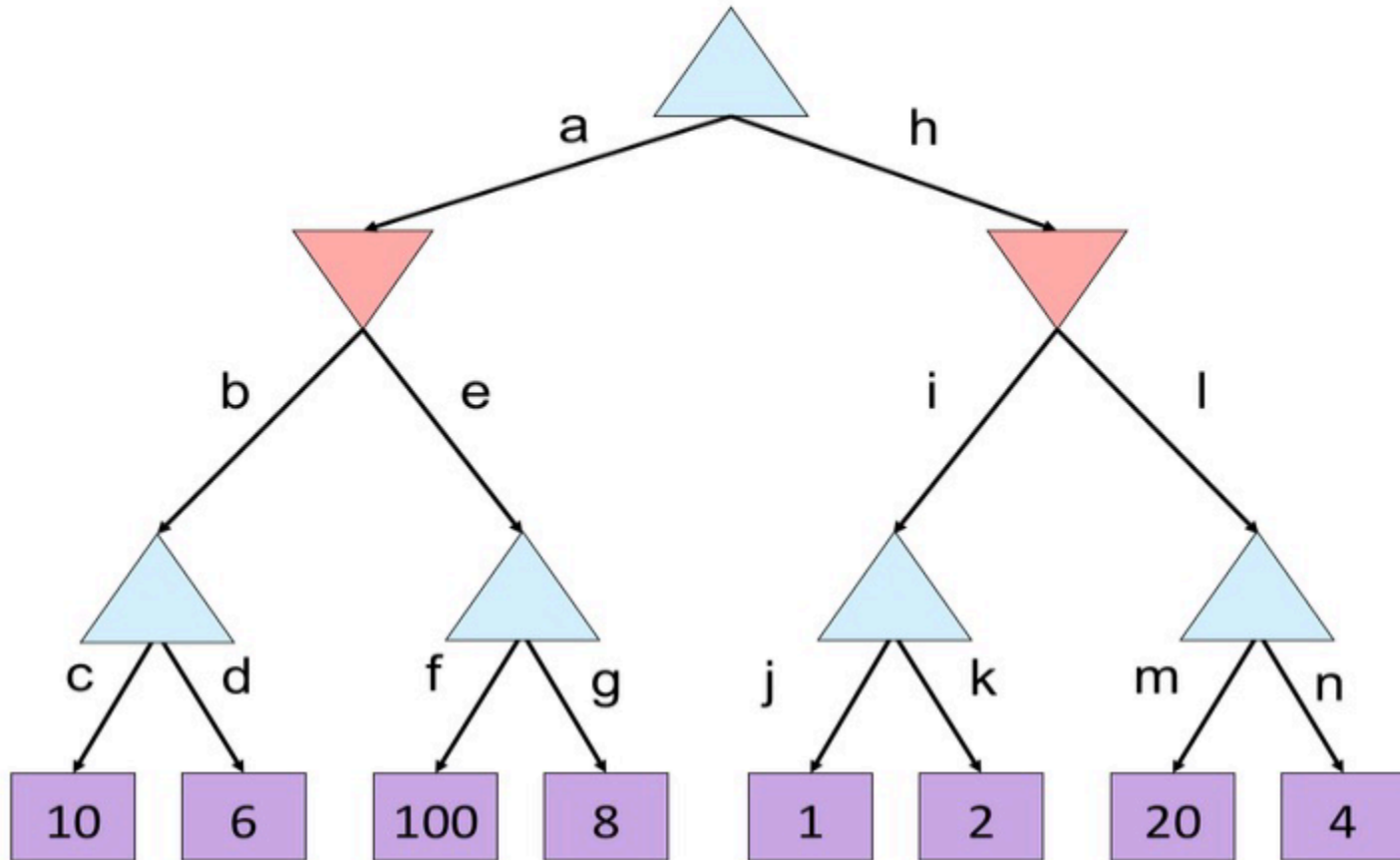


§ Este é um exemplo simples de metaraciocínio (calcular sobre o que calcular)

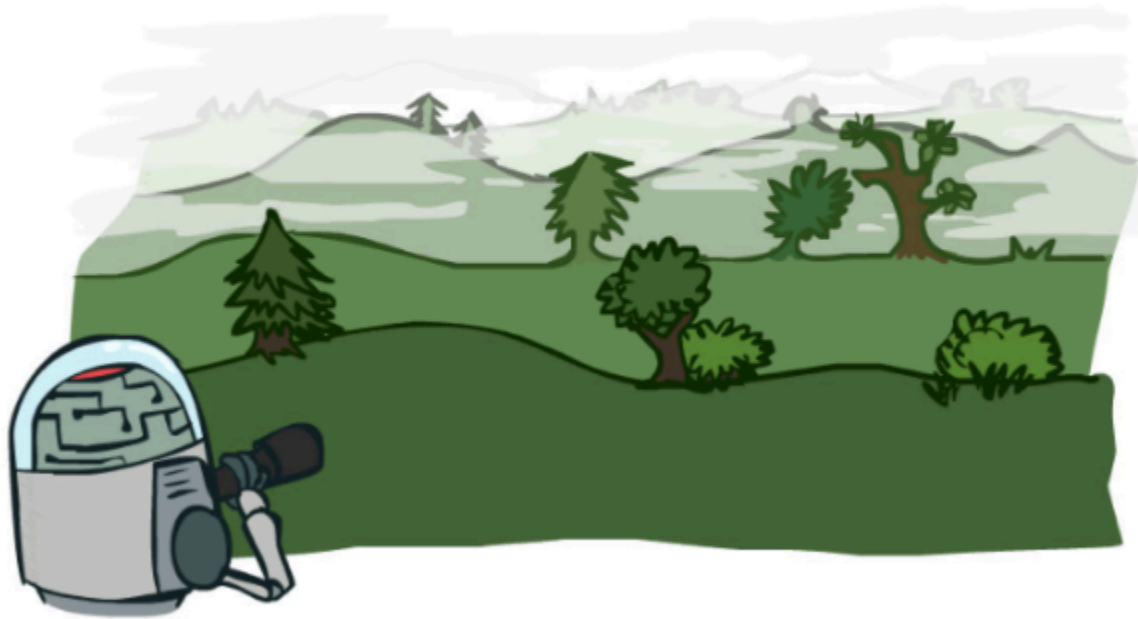
Alpha-Beta Quiz



Quiz Alpha-Beta 2

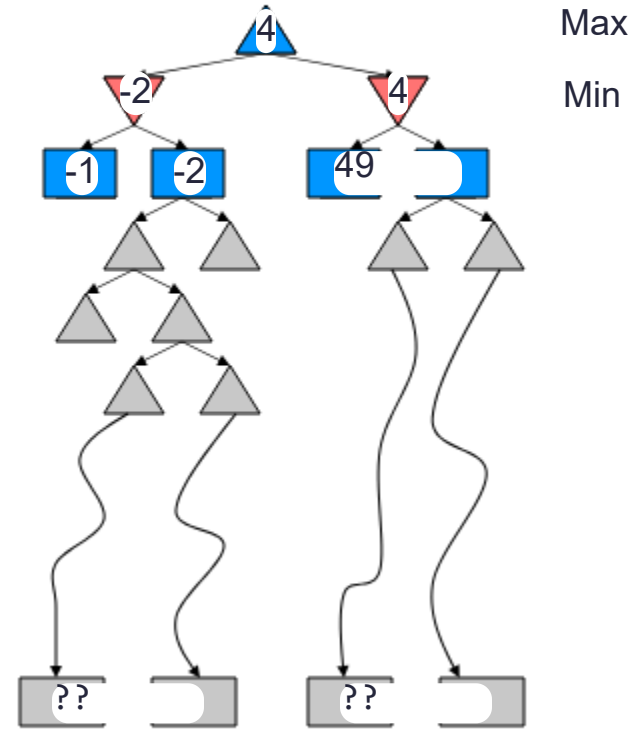


Limites de Recursos



Limites de Recursos

§ Problema: Em jogos realistas, não é possível buscar até as folhas! § Solução: busca limitada em profundidade § Em vez disso, busque apenas até uma profundidade limitada na árvore § Substitua utilidades terminais por uma função de avaliação para posições não terminais § Exemplo: § Suponha que temos 100 segundos, podemos explorar 10K nós / seg § Então podemos verificar 1M nós por lance § a-b atinge cerca de profundidade 8 – programa de xadrez decente § A garantia de jogo ótimo desaparece § Mais plies faz uma GRANDE diferença § Use aprofundamento iterativo para um algoritmo anytime

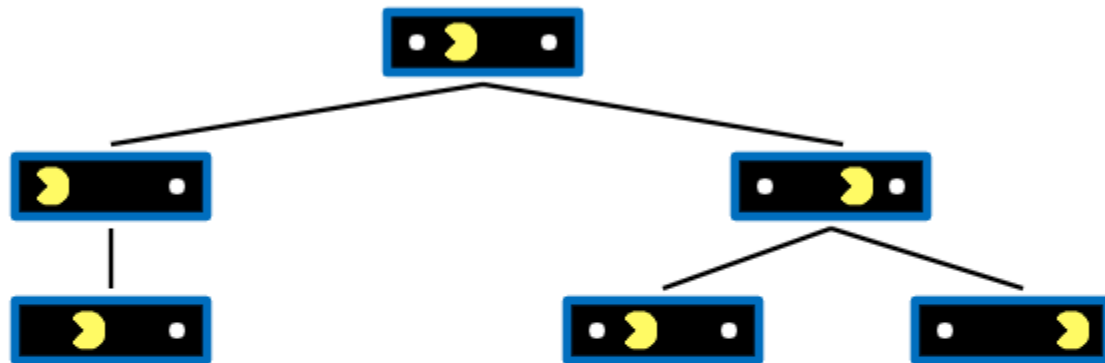


Vídeo de Demo Thrashing (d=2)



[Demo: bater d=2, bater d=2 (função de avaliação fixa) (L6D6)]

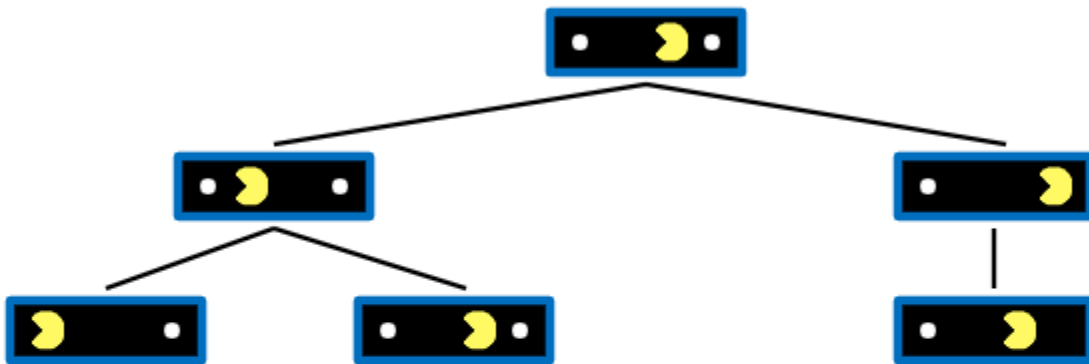
Por que o Pacman passa fome



§ Um perigo de agentes de replanejamento!

§ Ele sabe que sua pontuação vai subir ao comer o ponto agora (oeste, leste) § Ele sabe que sua pontuação vai subir tanto comendo o ponto depois (leste, oeste) § Não há oportunidades de pontuação depois de comer o ponto (no horizonte, dois aqui) § Portanto, esperar parece tão bom quanto comer: ele pode ir para o leste, Depois, de volta para o oeste na próxima rodada de replanejamento!

Por que o Pacman passa fome



§ Um perigo de agentes de replanejamento!

§ Ele sabe que sua pontuação vai subir ao comer o ponto agora (oeste, leste) § Ele sabe que sua pontuação vai subir tanto comendo o ponto depois (leste, oeste) § Não há oportunidades de pontuação depois de comer o ponto (no horizonte, dois aqui) § Portanto, esperar parece tão bom quanto comer: ele pode ir para o leste, Depois, de volta para o oeste na próxima rodada de replanejamento!

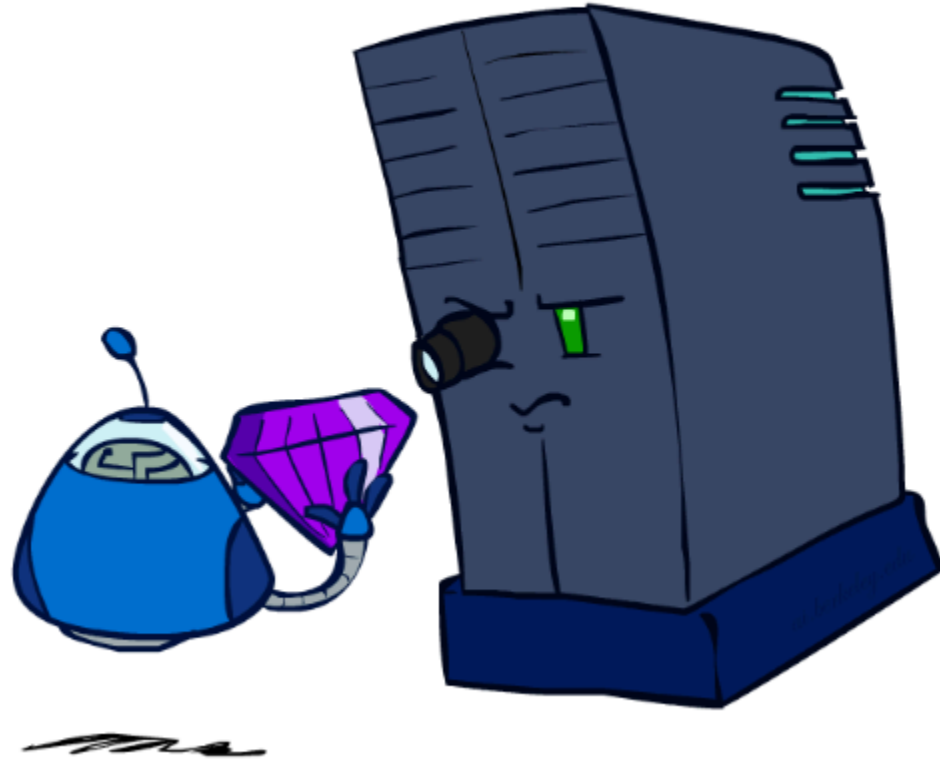
Vídeo do Demo Thrashing -- Corrigido

(d=2)



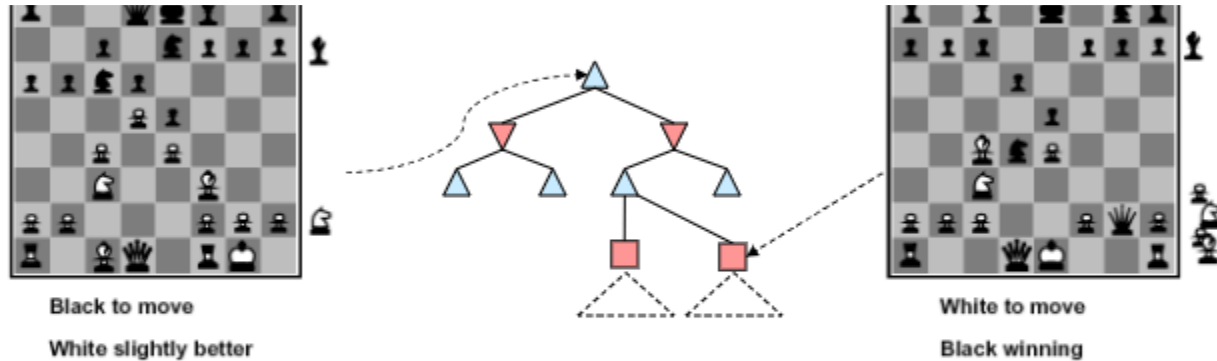
[Demo: bater d=2, bater d=2 (função de avaliação fixa) (L6D7)]

Funções de Avaliação



Funções de Avaliação

§ Funções de avaliação pontuam não-terminais em busca com profundidade limitada

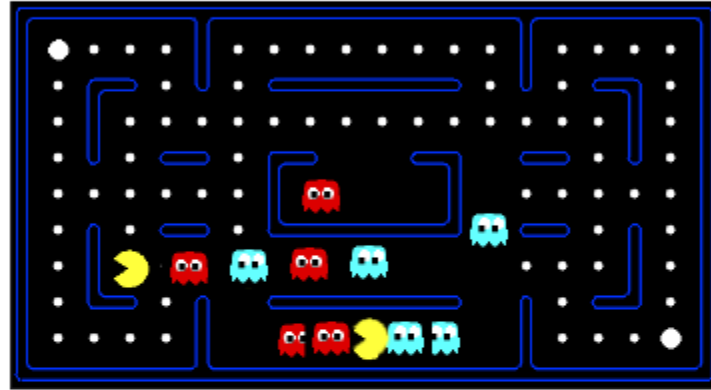


§ Função ideal: retorna o valor minimax real da posição
§ Na prática: tipicamente soma linear ponderada das características:

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

§ e.g. $f_1(s) = (\text{num white queens} - \text{num black queens})$, etc.

Avaliação para Pacman



[Demo: bater d=2, bater d=2 (função de avaliação fixa), fantasmas inteligentes coordenam (L6D6,7,8,10)]

Vídeo de Demonstração de Smart Ghosts (Coordenação)



Vídeo de demonstração de Smart Ghosts (Coordenação) – Ampliado



A profundidade importa

§ As funções de avaliação são sempre imperfeitas
§ Quanto mais fundo na árvore a função de avaliação está enterrada, menos importa a qualidade da função de avaliação
§ Um exemplo importante do trade-off entre complexidade de características e complexidade de computação



Vídeo da Demonstração Limitada de Profundidade (2)



Vídeo da Demonstração Limitada de Profundidade (10)



Sinergias entre a Função de Avaliação e a Alfa-Beta?

§ Alfa-Beta: a quantidade de poda depende da ordem de expansão

§ A função de avaliação pode fornecer orientação para expandir primeiro os nós mais promissores (o que depois torna mais provável que já exista uma boa alternativa no caminho para a raiz)

§ (um pouco semelhante ao papel da heurística A^* , filtragem de CSPs)

§ Alpha-Beta: (semelhante para papéis de min-max trocado)

§ O valor em um min-node só continuará descendo§ Uma vez que o valor do min-node é menor do que a opção melhor para o máximo ao longo do caminho até a raiz, pode podar§ Portanto: SE a função de avaliação fornece limite superior no valor no nó min, e o limite superior já é menor que a opção melhor para o máximo ao longo do caminho até a raizENTÃO pode podar

Próxima vez: Incerteza!
